SCHOOL OF INFORMATION TECHNOLOGY · KMUTT · SENIOR PROJECT CSC499

SmartFin: Automatic Expense Tracking with Thai E-Slip Recognition and an AI Financial Coach

สมาร์ทฟิน: ระบบบันทึกค่าใช้จ่ายอัตโนมัติด้วยการอ่านสลิปอิเล็กทรอนิกส์และผู้ช่วยทางการเงินอัจฉริยะ

Project Proposal

Submitted 18 September 2026

Student

Thaw Zin Moe Myint · Student ID [student ID]

Project Advisor

[advisor name and title]

Programme

[programme / curriculum] , Academic Year [year]

1 Introduction and Motivation

Cashless payment is now the default in Thailand. PromptPay transfers, QR payments and mobile banking applications such as K PLUS, SCB EASY, Krungthai NEXT and Bangkok Bank produce an electronic confirmation slip for every transaction, and most people keep those slips as images in their phone's photo gallery. The record of what a person spent therefore already exists on their device — but it exists as pictures, not as data that can be summed, categorised or questioned.

Personal finance applications available today do not close that gap. They ask the user to type each expense by hand, which is the step most people abandon within a few weeks. The applications that do offer automation are built around credit-card statements and bank feeds common in Western markets, and they handle neither the Thai-language merchant names nor the Buddhist-era dates (2567, 2569) printed on Thai slips. The result is a familiar pattern: a user installs a budgeting application, enters expenses diligently for a fortnight, and then stops.

SmartFin addresses this from the opposite direction. Rather than asking the user to record what they spent, it reads the evidence they already have. A payment slip is captured — by camera, or detected automatically in the gallery folder the banking application saves to — and a vision model extracts the merchant, the amount and the date, which the user confirms before the expense is stored. On top of that record, a conversational financial coach answers questions about the user's own spending by retrieving and computing over their stored transactions rather than by guessing.

2 Problem Statement

Three specific problems motivate this work.

1. **Manual entry is the point of failure in expense tracking.** Recording an expense requires the user to stop, open an application, and type an amount, a merchant and a category. The effort is small but it recurs several times a day, and compliance decays quickly.
2. **Thai payment slips are not served by existing extraction tools.** A Thai e-slip mixes Thai and Latin script, prints amounts with a ฿ symbol and satang decimals, and frequently dates transactions in the Buddhist era. Generic receipt-scanning services misread all three.
3. **Financial advice in existing applications is generic.** Applications that offer "insights" typically render fixed rules — a bar turning red at 80% of a budget — rather than answering a question about the user's actual data. A user cannot ask "how much did I spend on food in March?" and receive a figure computed from their own transactions.

3 Objectives

1. To design and implement a mobile application that records an expense from a Thai mobile-banking e-slip or a retail receipt with no manual typing of the merchant, amount or date.
2. To implement automatic detection of newly saved payment slips in the device photo gallery, so that an expense can be recorded without the user opening the application first.

3. To implement a conversational financial coach that answers questions about the user's own financial data using an agentic retrieval-augmented generation (RAG) architecture, in which a language model calls retrieval and action tools over the user's database records.
4. To implement a subscription and bill tracker that reminds the user of recurring payments before they fall due.
5. To evaluate both artificial-intelligence components quantitatively — the slip extractor against hand-labelled ground truth, and the coach against values recomputed from the database and against baseline architectures.

4 Scope and Limitations

4.1 In scope

- A mobile application for Android and iOS, built from a single Flutter codebase.
- Extraction of merchant, amount and date from Thai mobile-banking e-slips (PromptPay, K PLUS, SCB EASY, Krungthai NEXT, Bangkok Bank) and from printed retail receipts, including Buddhist-era date conversion.
- Expense and income records, categorisation, monthly summaries and a spending history with server-side filtering.
- A conversational coach with retrieval tools over the user's data, a small number of write actions, and a web-search tool for general financial questions.
- Manual entry of recurring bills, with due-date tracking and on-device reminders.
- Thai and English conversation.

4.2 Out of scope

- **No direct bank integration.** SmartFin does not connect to any banking API and never handles credentials. It reads only images the user already holds.
- **No automatic posting of expenses.** Every extracted slip is presented for confirmation before it is stored; the model's output is never written to the database unreviewed.
- No multi-currency support. All amounts are integer Thai Baht.
- No investment, lending, tax or credit advice.
- No shared or family accounts; each account is single-user.

4.3 Known limitations

- Extraction accuracy depends on a third-party vision model and on image quality. Measured accuracy is reported in Section 8.
- Automatic gallery detection depends on the folder in which a banking application saves its slips; users whose folder is not recognised must select it manually.
- Slip images are transmitted to a third-party model provider for extraction. This is disclosed to the user and discussed in Section 9.

5 Related Work and Technology Review

5.1 Existing personal finance applications

Commercial applications fall into two groups. The first — including the majority of applications marketed in Thailand — are manual ledgers with charts; they are reliable but depend entirely on user discipline. The second group automates ingestion through bank-feed aggregation, which is mature in markets with open-banking regulation but is not generally available to a student project in Thailand, and would in any case require handling banking credentials. SmartFin takes a third path: it automates ingestion from artefacts the user already possesses, which requires no institutional access and no credentials.

5.2 Optical extraction from receipts

Traditional receipt processing pairs an optical character recognition engine with rules or a trained parser to locate fields. This pipeline is brittle on Thai slips, where layout varies by bank and the script mixes with Latin digits. Recent multimodal large language models accept an image directly and return structured fields, absorbing layout variation without per-bank templates. SmartFin adopts this approach, prompting the model to return a JSON object and to convert Buddhist-era years explicitly.

5.3 Retrieval-augmented generation and agentic architectures

Retrieval-augmented generation supplies a language model with retrieved context so that its answers rest on data rather than on parametric memory. The conventional form retrieves once and generates once. An *agentic* variant instead gives the model a set of callable tools and lets it decide, over several turns, which to call and with what arguments — so a question requiring two different lookups, or a lookup followed by a write, can be served by one request. This project implements the agentic form and, in Section 8, measures it against the conventional one-shot form and against a no-retrieval baseline on the same task set.

6 Methodology and System Design

6.1 Development approach

The project follows an iterative approach with working software at each departmental milestone (Alpha, Beta, D-Day). Each feature is implemented end to end — client, API and database — before the next is started, so that every milestone has a demonstrable application rather than partial layers.

6.2 Technology stack

Layer	Technology	Rationale
Mobile client	Flutter (Dart 3.11)	One codebase for Android and iOS; direct access to camera and photo library.
Backend API	Go 1.26 with Fiber v2	Compiled, statically typed, low memory footprint; straightforward concurrency for multi-turn model calls.
Database	MongoDB Atlas	Document model suits variable transaction and summary shapes; managed hosting.
Vision extraction	Gemini 2.5 Flash	Accepts an image directly and returns structured JSON; handles Thai script without a separate OCR stage.
Coach reasoning	Gemini 2.5 Flash function calling	Native tool-calling loop, which is the mechanism the agentic architecture requires.
Authentication	Firebase Authentication	Email and Google sign-in without implementing credential storage.
Notifications	On-device local notifications	Bill reminders without a server-side scheduler.

Table 1 — Implementation technologies.

6.3 System architecture

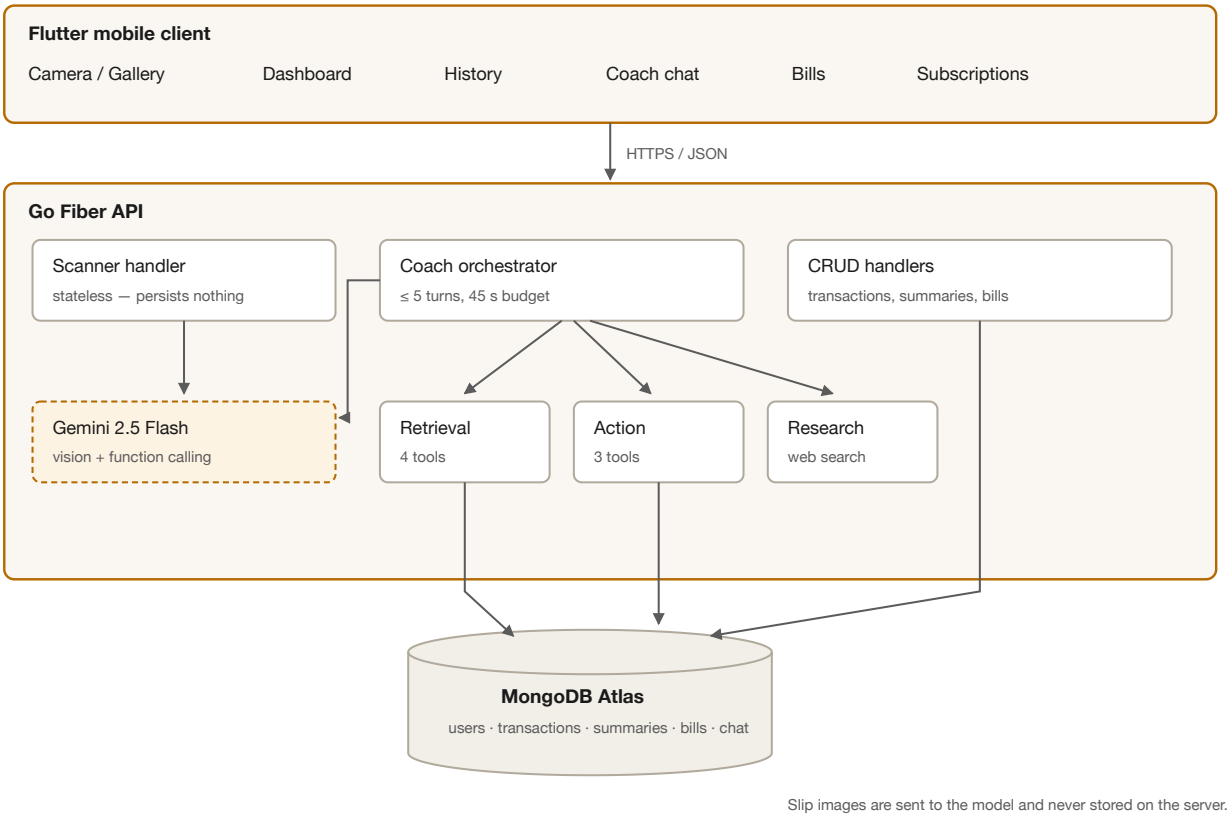


Figure 1 — System architecture. Arrows show the direction of calls.

Two design decisions in this architecture are worth stating explicitly. First, **slip scanning is stateless on the server**: the extraction endpoint returns fields and persists nothing, so the transaction is created only when the user confirms it on the verification screen. Second, **all**

monetary values are integer Thai Baht end to end, from the database through the API to the interface, so that no floating-point rounding can alter a user's recorded amount.

6.4 Slip capture flow

The application offers two paths to the same outcome. In the *manual* path the user photographs a slip or receipt. In the *automatic* path a gallery watcher detects a newly saved image in a folder associated with a banking application and offers it. Both paths converge on an extraction request, followed by a verification screen where the user checks the merchant, amount, date and category before saving. Because folder naming varies between banking applications and devices, folder names are matched after normalisation, several folders are watched simultaneously, and the user may nominate a folder manually when automatic matching fails.

6.5 The AI coach

A question sent to the coach starts a bounded reasoning loop on the server. The model receives the user's question, the current date in Thai local time, and a set of tool declarations grouped into three roles:

- **Retrieval tools** — monthly budget summary, transactions (filterable by category, type and merchant), subscriptions, and recent conversation history.
- **Action tools** — set a savings goal, update a budget limit, add a transaction.
- **Research tool** — web search, for general financial questions not answerable from the user's own data.

The model selects tools, receives their results, and may call further tools before composing an answer. The loop is bounded at five turns and a 45-second server budget. Every exchange is persisted so that the conversation survives the client screen being disposed, which also gives the coach a memory of earlier sessions.

7 Evaluation Methodology

Both artificial-intelligence components are evaluated quantitatively against ground truth rather than by demonstration or by subjective judgement. An earlier attempt used a language model to judge its own answers; it was discarded because the judge scored against a hint of the context rather than against the real data, so its scores could not be defended. Both current harnesses are deterministic and scored by rule.

7.1 Coach evaluation

A test set of 30 scripted tasks spans ten categories: lookup, aggregation, bills, Thai-language questions, multi-turn conversation, empty-account behaviour, off-topic redirection, safety, write actions and memory. Each task runs on a freshly seeded account, and the expected values are recomputed from MongoDB rather than hard-coded, so the harness cannot drift from the data. Answers are scored by rule: required figures must appear within ±฿1, required keywords must be present, prohibited keywords must be absent, and for write tasks the database is read back to confirm the change actually occurred. Three architectures are compared on identical tasks:

- **Agentic RAG** — the implemented system, with the full tool loop.
- **One-shot RAG** — the same model and persona, given the user's data as context in a single turn, with no tools.
- **No retrieval** — the same model and persona with no data at all, to establish a floor.

Every task is repeated three times per architecture, giving 90 trials per architecture. Two figures are reported: the pass rate on the 23 tasks every architecture is capable of attempting, and overall task success on all 30 — where an architecture that cannot perform a task fails it.

7.2 Slip extraction evaluation

Thirty real images — 15 mobile-banking e-slips and 15 printed retail receipts — were labelled by hand with the true merchant, amount and date, then scanned twice each through the production endpoint, giving 60 scans. An amount is correct within ±0.5 Baht; a merchant is correct if the extracted name contains an accepted alias, which accommodates transliteration; a date must match exactly. Where a receipt prints no date at all, the extractor is required to return the current date rather than invent one. Because the images are real payment records containing personal data, they are excluded from version control and no identifying detail is reproduced in any report.

8 Results to Date

8.1 Coach evaluation results

Metric	Agentic RAG	One-shot RAG	No retrieval
Pass rate, 23 shared questions (69 trials)	100% (69/69)	100% (69/69)	—
Overall task success, all 30 tasks (90 trials)	**97% (87/90)**	87% (78/90)	17%
Write actions performed	7/9	0/9	0/9
Cross-session memory	2/3	0/3	0/3
Numeric accuracy	100%	100%	—
Unsupported ฿ figures (fabrication)	0%	0%	—
Median response time	3.2 s	2.0 s	—

Table 2 — Coach architectures on 30 tasks × 3 repetitions (90 trials each).

Three findings deserve emphasis, including one that does not favour the implemented system.

- **Retrieval is what matters most.** Without access to the user's data the model answers 17% of tasks; with it, both retrieval architectures answer every shared question correctly. This is the difference between an application that knows the user's finances and one that does not.
- **One-shot RAG matches the agentic system on read-only questions**, and is faster. The agentic architecture earns its cost on the tasks the baseline structurally cannot do: it performed 7 of 9 write actions and 2 of 3 memory tasks, where the baseline scored zero on both — and in 7 of 9 write trials the baseline *claimed* to have performed an action it had no ability to perform.
- **Neither retrieval architecture fabricated a monetary figure** across 180 trials, which is the property that matters most in a financial application.

Two failure modes were found and corrected during evaluation: the transaction tool could not filter by merchant name, so merchant-specific questions failed; and the model sometimes replied "give me a moment" instead of calling a tool. Both were fixed and re-measured, raising the shared pass rate from 84% to 100%. Three agentic failures remain: two false "logged" claims after a write, and one missed memory recall.

8.2 Slip extraction results

Field	All images	E-slips (30 scans)	Receipts (30 scans)
Amount	**100% (60/60)**	100%	100%
Merchant	93% (56/60)	—	—
Date	93% (56/60)	—	—
All three fields correct	87% (52/60)	93% (28/30)	80% (24/30)

Table 3 — Extraction accuracy on 30 labelled images, scanned twice each (60 scans).

Amounts — the field with the greatest consequence if wrong — were extracted correctly in every scan, including satang decimals, rotated and creased receipts, and Buddhist-era years. The eight remaining

errors have two causes: four scans invented a date on two receipts that print no date, and four misread Thai shop names. Both are attributable to the extraction prompt rather than to the model's capability, and correcting them is scheduled before the Beta Test. Identical output was produced on both scans of 87% of images, and the median extraction time was 5.2 seconds. Measured token usage puts the cost of one scan at approximately ฿0.05.

> **On self-reported confidence.** The extractor was asked to return a confidence score, and returned 0.98 on correct and incorrect extractions alike. The score was therefore removed from the interface rather than shown to users, since a confident-looking wrong answer is worse than no score at all. The verification screen now asks every user to check every field.

9 Privacy and Ethical Considerations

- **Payment slips contain personal data** — names, partial account numbers and transaction references. Slip images used for evaluation are excluded from version control, and no name taken from a slip appears in any report, slide or summary.
- **Images are processed by a third-party model provider.** This is inherent to the chosen approach and is disclosed to the user; images are not stored on the project's server, which holds only the confirmed fields.
- **No banking credentials are requested or handled at any point.**
- **The user confirms every extracted expense** before it is recorded, so no financial record is created by the model alone.
- **The coach declines actions outside its remit**, including deletion requests, and its write actions are limited to three narrowly defined operations.

10 Project Timeline

Phase	Date	Deliverable	Status
Core implementation	To 14 Sep 2026	Authentication, transactions, dashboard, history, slip scanning, AI coach, bill tracker	Complete
Alpha Test	14 Sep 2026	Working application demonstrated; both AI components evaluated	Complete
Refinement	Sep – Oct 2026	Extraction prompt corrections, write-action verification, interface revisions from Alpha feedback	In progress
Beta Test	14 – 20 Oct 2026	Feature-complete application; user testing with real participants	Scheduled
Deployment and hardening	Oct – Nov 2026	Backend deployment, credential rotation, larger evaluation set	Scheduled
D-Day presentation	16 Nov 2026	Final demonstration and defence	Scheduled
Final report	18 Dec 2026	Complete project report	Scheduled
Project video	23 Dec 2026	Demonstration video	Scheduled

Table 4 — Milestones. Departmental milestone dates as scheduled for CSC499.

11 Expected Outcomes

1. A working mobile application that records a Thai payment slip as a categorised expense without manual typing, with measured field accuracy reported rather than asserted.
2. A conversational financial coach that answers questions from the user's own data, with a quantified comparison against two baseline architectures on an identical task set.
3. Two reusable evaluation harnesses — one for extraction accuracy against labelled images, one for coach answers against database ground truth — that can be re-run after any change to a prompt, a tool or a model.
4. Evidence on a question of practical interest beyond this project: *when* an agentic architecture is worth its additional latency and cost compared with conventional single-pass retrieval. The measurements here indicate the answer is: when the task requires acting or remembering, not when it requires only looking up.

12 References

1. Bank of Thailand. *Payment Systems Report*. Bangkok: Bank of Thailand. [Statistics on PromptPay and mobile-banking adoption.]
2. Lewis, P. et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 2020.
3. Yao, S. et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*, 2023.

4. Schick, T. et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems*, 2023.
5. Google. *Gemini API Documentation: Image Understanding and Function Calling*. Google AI for Developers.
6. Office of the Personal Data Protection Committee. *Personal Data Protection Act B.E. 2562 (2019)*. Bangkok.
7. [add any papers or applications your advisor expects to see cited]